Métodos de Análisis de Datos 1

Clase 5: Transformación de Datos

Universidad Nacional del Sur

Contenidos de hoy

¿Por qué importan los tipos de datos?

Problema común

Los datos se cargan con tipos incorrectos

Ejemplos

- Números como texto: `"123"` en vez de `123`
- Fechas como texto: `"2024-01-15"` en vez de `datetime`
- Categóricas como texto: `"rojo"` en vez de `category`
- Booleanos como números: `1/0` en vez de `True/False`

¿Por qué importan los tipos de datos?

Problema común

Los datos se cargan con tipos incorrectos

Ejemplos

- Números como texto: `"123"` en vez de `123`
- Fechas como texto: `"2024-01-15"` en vez de `datetime`
- Categóricas como texto: `"rojo"` en vez de `category`
- Booleanos como números: `1/0` en vez de `True/False`

Consecuencias

- Operaciones incorrectas (`"1" + "2" = "12"`)
- Más memoria usada
- Visualizaciones erróneas
- Modelos de ML fallan

Tipos de datos principales

Numéricos:

- `int`: Enteros
- `float`: Decimales

Texto:

- `string`: Texto general
- `category`: Categóricas (menos memoria)

Booleanos:

- `bool`: True/False

Fechas:

- `datetime`: Fecha y hora
- `date`: Solo fecha

Categóricos ordenados:

- `ordinal`: Con orden natural

Conversión de tipos en Python

```
import pandas as pd

# A num rico (errores      NaN)
df['edad'] = pd.to_numeric(df['edad'], errors='coerce')

# A string
df['id'] = df['id'].astype(str)

# A fecha
df['fecha'] = pd.to_datetime(df['fecha'], format='%Y-%m-%d')

# A categ rico
df['color'] = df['color'].astype('category')

# Categ rico ordenado
df['educacion'] = pd.Categorical(
    df['educacion'],
```

Conversión de tipos en R

```
library(lubridate)

# A num rico
df$edad <- as.numeric(df$edad)

# A string
df$id <- as.character(df$id)

# A fecha
df$fecha <- ymd(df$fecha)  # a o -mes-d a
# Otras: dmy(), mdy(), ymd_hms()

# A factor
df$color <- as.factor(df$color)

# Factor ordenado
df$educacion <- factor(
```

Feature Engineering básico

¿Qué son variables derivadas?

Nuevas variables creadas a partir de las existentes

¿Por qué crearlas?

- Extraer información útil
- Capturar relaciones no lineales
- Mejorar performance de modelos
- Facilitar interpretación

Feature Engineering básico

¿Qué son variables derivadas?

Nuevas variables creadas a partir de las existentes

¿Por qué crearlas?

- Extraer información útil
- Capturar relaciones no lineales
- Mejorar performance de modelos
- Facilitar interpretación

Ejemplos

- IMC = peso / altura^2
- Edad = hoy - fecha_nacimiento
- Nombre_completo = nombre + apellido
- Es_adulto = edad > 18

Variables derivadas: Python

```
# Operaciones aritm ticas
df['imc'] = df['peso'] / (df['altura'] ** 2)

# Operaciones con fechas
df['edad'] = (pd.Timestamp.now() - df['fecha_nac']).dt.days / 365.25

# Extraer componentes de fecha
df['a  o'] = df['fecha'].dt.year
df['mes'] = df['fecha'].dt.month
df['dia_semana'] = df['fecha'].dt.dayofweek  # 0=lunes

# Operaciones con texto
df['nombre_completo'] = df['nombre'] + ' ' + df['apellido']
df['longitud_nombre'] = df['nombre'].str.len()

# Condiciones
df['adulto'] = df['edad'] >= 18
```

Variables derivadas: R

```r
library(tidyverse)

df <- df %>%
  mutate(
    # Aritm tica
    imc = peso / (altura^2),

    # Fechas
    edad = as.numeric(difftime(Sys.Date(), fecha_nac, units = "days"
       )) / 365.25,
    a  o = year(fecha),
    mes = month(fecha),
    dia_semana = wday(fecha),  # 1=domingo

    # Texto
    nombre_completo = paste(nombre, apellido),
    longitud_nombre = nchar(nombre),
```

El problema de las variables categóricas

Algoritmos de ML requieren números

La mayoría de algoritmos no pueden trabajar directamente con texto

Ejemplo

Color
rojo
verde
azul

$\rightarrow$ ¿Cómo convertir a números?

El problema de las variables categóricas

Algoritmos de ML requieren números

La mayoría de algoritmos no pueden trabajar directamente con texto

Ejemplo

Color
rojo
verde
azul

$\rightarrow$ ¿Cómo convertir a números?

Dos estrategias principales

1. **Label Encoding:** Asignar un número a cada categoría
2. **One-Hot Encoding:** Crear columna binaria por categoría

Label Encoding

Concepto

Asigna un número único a cada categoría

Color	Codificado
rojo	0
verde	1
azul	2

Label Encoding

Concepto

Asigna un número único a cada categoría

Color	Codificado
rojo	0
verde	1
azul	2

Cuidado

Solo usar para variables ordinales (con orden natural)

Para variables nominales, el modelo asumirá que $2 > 1 > 0$, lo cual no tiene sentido para colores

¿Cuándo usar?

Educación: primaria (0) $<$ secundaria (1) $<$ universitaria (2)

Label Encoding: Código

Python:

```python
from sklearn.preprocessing import
    LabelEncoder

# Label encoding
le = LabelEncoder()
df['educacion_encoded'] = le.
    fit_transform(
     df['educacion']
)

# Ver mapeo
mapeo = dict(zip(
     le.classes_,
     le.transform(le.classes_)
))
print(mapeo)
```

R:

```r
# Autom tico con factor
df$educacion_encoded <- as.numeric
    (
  factor(df$educacion)
)

# Con orden espec fico
df$educacion_encoded <- as.numeric
    (
  factor(
    df$educacion,
    levels = c('primaria',
               'secundaria',
               'universitaria')
  )
)
```

One-Hot Encoding

Concepto

Crea una columna binaria (0/1) para cada categoría

Original	color_rojo	color_verde	color_azul
rojo	1	0	0
verde	0	1	0
azul	0	0	1
rojo	1	0	0

One-Hot Encoding

Concepto

Crea una columna binaria (0/1) para cada categoría

Original	color_rojo	color_verde	color_azul
rojo	1	0	0
verde	0	1	0
azul	0	0	1
rojo	1	0	0

Ventajas

- No asume orden entre categorías
- Funciona para variables nominales
- Fácil de interpretar

Desventaja

One-Hot Encoding: Python

```
import pandas as pd

# M todo simple
df_encoded = pd.get_dummies(
    df,
    columns=['color'],
    prefix='color'
)

# Con sklearn (m s control)
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(
    sparse=False,       # Retornar array denso
    drop='first'        # Eliminar primera categor a
)                       # (evita multicolinealidad)
```

One-Hot Encoding: R

```r
library(caret)

# Con caret
dummies <- dummyVars(" ~ color", data = df)
df_encoded <- predict(dummies, newdata = df)

# Con tidyverse
df_encoded <- df %>%
  mutate(value = 1) %>%
  pivot_wider(
    names_from = color,
    values_from = value,
    values_fill = 0,
    names_prefix = "color_"
  )
```

¿Label o One-Hot?

Label Encoding:

- Variables ordinales
- Árboles de decisión
- Menos columnas
- Variables nominales
- Regresión lineal

One-Hot Encoding:

- Variables nominales
- Regresión lineal/logística
- Neural networks
- Muchas categorías
- Aumenta dimensionalidad

Regla práctica

- Ordinal $\rightarrow$ Label Encoding
- Nominal con pocas categorías (<10) $\rightarrow$ One-Hot
- Nominal con muchas categorías $\rightarrow$ Target Encoding, Frequency Encoding, etc.

Pipeline de transformación

1. **Verificar tipos de datos**
   - `df.dtypes` (Python) o `str(df)` (R)
2. **Convertir tipos incorrectos**
   - Fechas como datetime
   - Números como int/float
   - Categóricas como category/factor
3. **Crear variables derivadas**
   - Extraer info de fechas
   - Combinar columnas
   - Aplicar fórmulas
4. **Codificar categóricas**
   - Label encoding para ordinales
   - One-hot encoding para nominales

Resumen de la clase

- Los **tipos de datos** deben ser correctos
- Conversión en Python: `astype()`, `pd.to_numeric()`, `pd.to_datetime()`
- Conversión en R: `as.numeric()`, `ymd()`, `factor()`
- Las **variables derivadas** extraen información útil
- **Label encoding:** solo para ordinales
- **One-hot encoding:** para nominales
- La elección depende del tipo de variable y del algoritmo

Próxima clase

Normalización, estandarización e integración de datos

Tarea

1. Usar el dataset limpio de la clase anterior
2. Transformaciones a realizar:
   - Verificar y corregir tipos de datos
   - Crear al menos 3 variables derivadas útiles
   - Si hay fechas, extraer componentes (año, mes, día semana)
   - Identificar variables categóricas
   - Codificar variables categóricas apropiadamente
3. Documentar en el notebook:
   - Qué transformaciones hiciste
   - Por qué elegiste label o one-hot
   - Comparar dimensiones antes vs después
4. Subir a GitHub

¿Preguntas?